

On This Page >

Introduction to Vars

The vars system is clan's declarative solution for managing generated files, secrets, and dynamic configuration in your NixOS deployments. It eliminates the manual steps of generating credentials, certificates, and other dynamic values by automating these processes within your infrastructure-as-code workflow.

What Problems Does Vars Solve?

Before Vars: Manual Secret Management

Traditional NixOS deployments require manual steps for secrets and generated files:

```
1 # Generate password hash manually
2 mkpasswd -m sha-512 > /tmp/root-password-hash
3 # Copy hash into configuration
4 users.users.root.hashedPasswordFile = "/tmp/root-password-hash";
```

This approach has several problems:

- **Not reproducible:** Manual steps vary between team members
- **Hard to maintain:** Updating secrets requires remembering manual commands
- **Deployment friction:** Secrets must be managed outside of your configuration
- **Team collaboration issues:** Sharing credentials securely is complex

After Vars: Declarative Generation

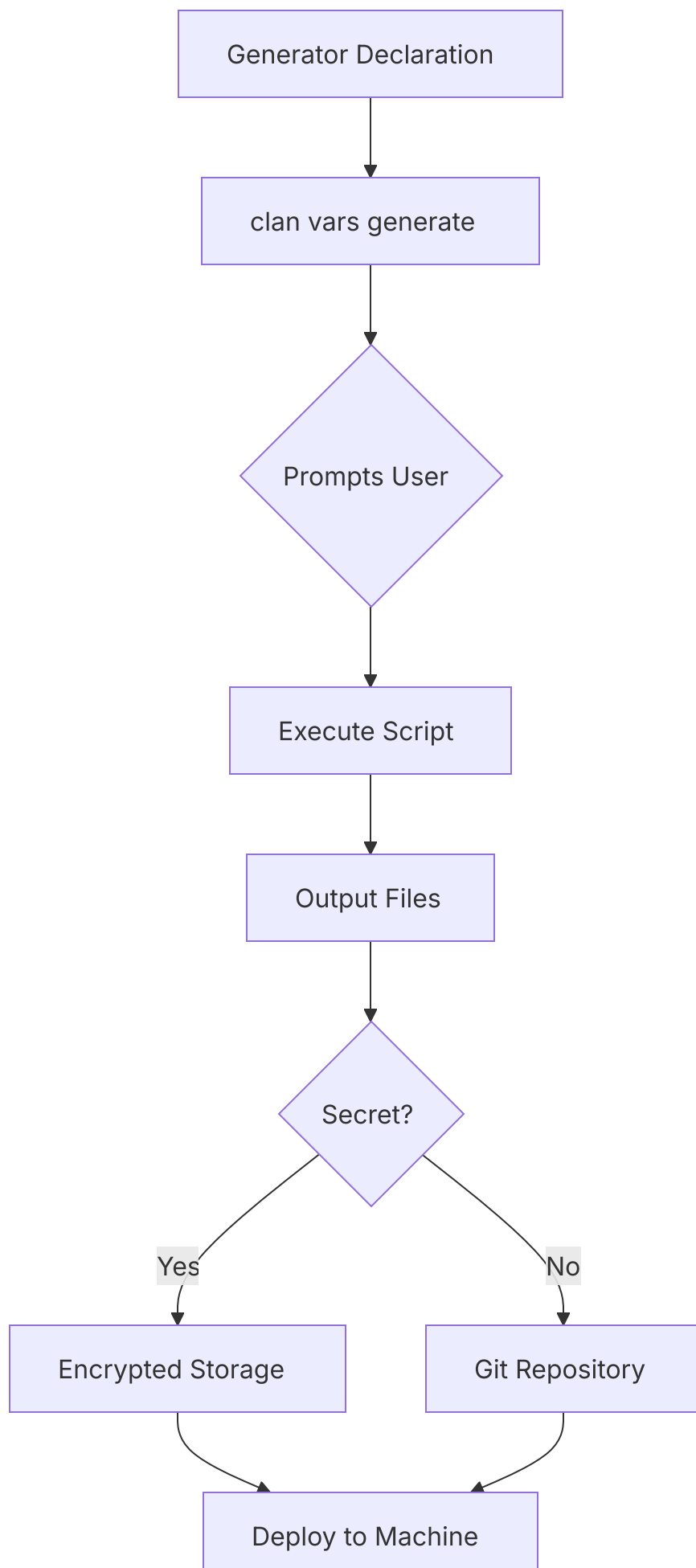
With vars, the same process becomes declarative and automated:

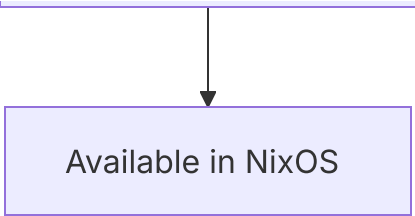
```
1 clan.core.vars.generators.root-password = {  
2   prompts.password.description = "Root password";  
3   prompts.password.type = "hidden";  
4   files.hash.secret = false;  
5   script = "mkpasswd -m sha-512 < $prompts/password > $out/hash";  
6   runtimeInputs = [ pkgs.mkpasswd ];  
7 };  
8  
9 users.users.root.hashedPasswordFile =  
10  config.clan.core.vars.generators.root-password.files.hash.path;
```

Core Benefits

-  **Reproducible:** Same inputs always produce the same outputs
-  **Declarative:** Defined alongside your NixOS configuration
-  **Secure:** Automatic secret storage and encrypted deployment
-  **Collaborative:** Built-in sharing for team environments
-  **Automated:** No manual intervention required for deployments
-  **Integrated:** Works seamlessly with clan's deployment workflow

How It Works





- 1. **Declare generators** in your NixOS configuration
- 2. **Generate values** using `clan vars generate` (or automatically during deployment)
- 3. **Store securely** in encrypted backends or version control
- 4. **Deploy seamlessly** to your machines where they're accessible as file paths

Common Use Cases

Use Case	What Gets Generated	Benefits
User passwords	Password hashes	No plaintext in config
SSH keys	Host/user keypairs	Automated key rotation
TLS certificates	Certificates + private keys	Automated PKI
Database credentials	Passwords + connection strings	Secure service communication
API tokens	Random tokens	Service authentication
Configuration files	Complex configs with secrets	Dynamic config generation

Architecture Overview

The vars system has three main components:

1. Generators

Define how to create files from inputs:

- **Prompts:** Values requested from users
- **Scripts:** Generation logic

- **Dependencies:** Other generators this depends on
- **Outputs:** Files that get created

2. Storage Backends

Handle secret storage and deployment:

- **sops:** Encrypted files in git (recommended)
- **password-store:** GPG/age-based secret storage

Quick Start Example

Here's a complete example showing password generation and usage:

```
1 # generator.nix
2 { config, pkgs, ... }: {
3   clan.core.vars.generators.user-password = {
4     prompts.password = {
5       description = "User password";
6       type = "hidden";
7     };
8     files.hash = { secret = false; };
9     script = ''
10       mkpasswd -m sha-512 < $prompts/password > $out/hash
11     '';
12     runtimeInputs = [ pkgs.mkpasswd ];
13   };
14
15   users.users.myuser = {
16     hashedPasswordFile =
17       config.clan.core.vars.generators.user-password.files.hash.path;
18   };
19 }
```

```
1 # Generate the password
2 clan vars generate my-machine
3
4 # Deploy to machine
5 clan machines update my-machine
```

Migration from Facts

If you're currently using the legacy facts system, see our [Migration Guide](#) for step-by-step instructions on upgrading to vars.

← Previous

Internal Services with SSL

Next →

Minimal Example

Need help? Reach out to [our community](#).